

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №2
дисциплины «Искусственный интеллект и машинное обучение

Вариант 20

Выполнил:
Скоробогатов Виктор Андреевич
2 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Руководитель практики:
Воронкин Роман Александрович,
доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: Основы работы с библиотекой NumPy

Цель: исследовать базовые возможности библиотеки NumPy языка программирования Python

Ход работы

Ссылка на репозиторий:

<https://github.com/LostsSs78/AI-Lab2>

Для выполнения работы был создан новый общедоступный репозиторий на GitHub, использующий лицензию MIT и язык программирования Python, далее он был клонирован на персональный компьютер, также был дополнен файл .gitignore необходимыми правилами для языка Python, IDE PyCharm и Jupyter Lab:

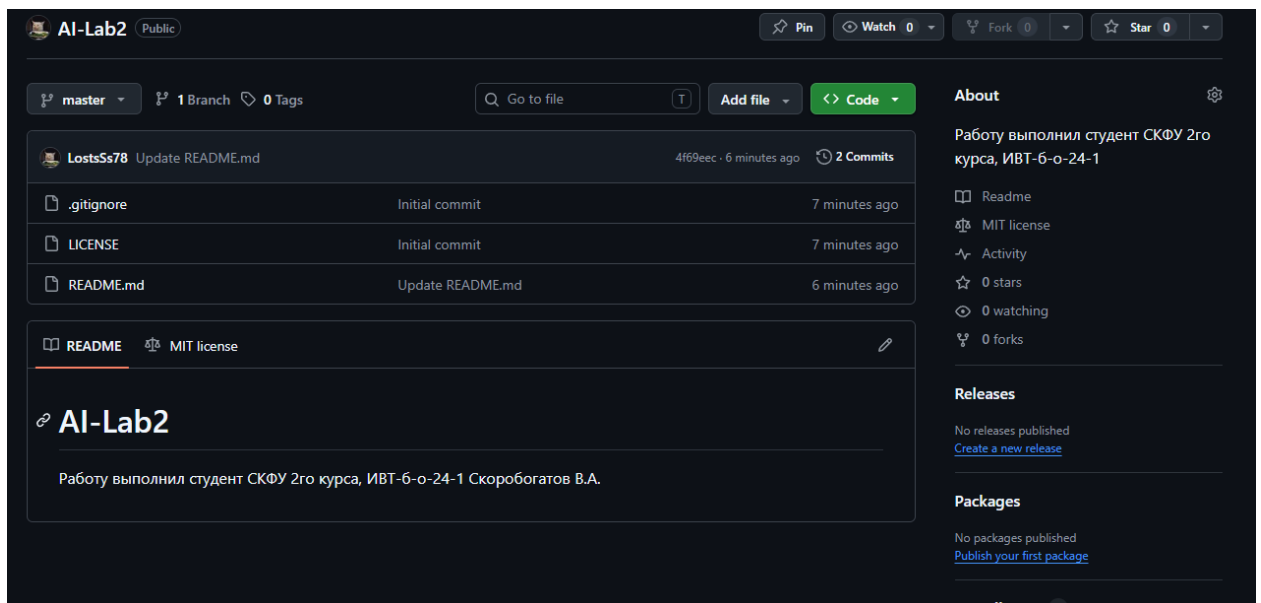


Рисунок 1. Репозиторий на GitHub

В ходе работы были проработаны примеры задач:

```
Пример 1. Массивы в NumPy

[7]: import numpy as np

      a = np.array([1, 2, 3])
      b = np.array([4, 5, 6])

      result = a + b # Быстрое сложение массивов
      print(result)

[5 7 9]
```

Рисунок 2. Пример 1

```
Пример 2. Доступ к элементам массива

[24]: m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')

      print(m)
      print(m[1, 0])
      print(m[1, :])
      print(m[:, 2])
      print(m[1, :2])

[[1 2 3 4]
 [5 6 7 8]
 [9 1 5 7]]
5
[[5 6 7 8]]
[[3]
 [7]
 [5]]
[[5 6]]
```

Рисунок 3. Пример 2

```
Пример 3. Расчет статистик по данным в массиве

[20]: m = np.matrix('1 2 3 4; 5 6 7 8; 9 1 5 7')
      print(m)
      type(m)
      m = np.array(m)
      type(m)
      print(m.shape)
      print(m.max())
      print(m.max(axis=1))
      print(m.max(axis=0))
      print(m.mean())
      print(m.mean(axis=1))
      print(m.sum())

[[1 2 3 4]
 [5 6 7 8]
 [9 1 5 7]]
(3, 4)
9
[4 8 9]
[9 6 7 8]
4.833333333333333
[2.5 6.5 5.5]
58
```

Рисунок 4. Пример 3

Пример 4 Использование boolean массива для доступа к ndarray

```
[23]: nums = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
      letters = np.array(['a', 'b', 'c', 'd', 'a', 'e', 'b'])

      nums = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
      less_than_5 = nums < 5

      print(less_than_5)

      pos_a = letters == 'a'
      print(pos_a)

[ True  True  True  True False False False False False]
[ True False False False  True False False]
```

Рисунок 5. Пример 4

Пример 5. Доп. функции: arange

```
[26]: print(np.arange(10))

      print(np.arange(5, 12))

      print(np.arange(1, 5, 0.5))

[0 1 2 3 4 5 6 7 8 9]
[ 5  6  7  8  9 10 11]
[1.  1.5 2.  2.5 3.  3.5 4.  4.5]
```

Пример 6 Доп. функции: matrix

```
[29]: a = [[1, 2], [3, 4]]
      print(np.matrix(a))
      print("-"*20)

      # Вариант с массивом NumPy.

      b = np.array([[5, 6], [7, 8]])
      print(np.matrix(b))
      print("-"*20)

      # Вариант в Matlab стиле.

      print(np.matrix('[1, 2; 3, 4]'))

[[1 2]
 [3 4]]
-----
[[5 6]
 [7 8]]
-----
[[1 2]
 [3 4]]
```

Рисунок 6. Примеры 5-6

```
Пример 12 Матрицы. Вектор-строка

[43]: v_hor_np = np.array([1, 2])
      print(v_hor_np)
      print("-"*20)

      v_hor_zeros_v1 = np.zeros((5,))
      print(v_hor_zeros_v1)
      print("-"*20)

      v_hor_zeros_v2 = np.zeros((1, 5))
      print(v_hor_zeros_v2)
      print("-"*20)

      v_hor_one_v1 = np.ones((5,))
      print(v_hor_one_v1)
      print("-"*20)

      v_hor_one_v2 = np.ones((1, 5))
      print(v_hor_one_v2)

[1 2]
-----
[0. 0. 0. 0. 0.]
-----
[[0. 0. 0. 0. 0.]]
-----
[1. 1. 1. 1. 1.]
-----
[[1. 1. 1. 1. 1.]]
```

Рисунок 7. Пример 12

```
Пример 14 Квадратная матрица

[46]: m_sqr_arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
      print(m_sqr_arr)
      print("-"*20)

      m_sqr = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
      m_sqr_arr = np.array(m_sqr)
      print(m_sqr_arr)
      print("-"*20)

      m_sqr_mx = np.matrix([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
      print(m_sqr_mx)

[[1 2 3]
 [4 5 6]
 [7 8 9]]
-----
[[1 2 3]
 [4 5 6]
 [7 8 9]]
-----
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Рисунок 8. Пример 14

Пример 16 Транспонирование матрицы

```
[50]: A = np.matrix('1 2 3; 4 5 6')
      print(A)
      print("-"*20)

      A_t = A.transpose()
      print(A_t)
      print("-"*20)

      print(A.T)

[[1 2 3]
 [4 5 6]]
-----
[[1 4]
 [2 5]
 [3 6]]
-----
[[1 4]
 [2 5]
 [3 6]]
```

Рисунок 9. Пример 16

Пример 26 Ранг матрицы

```
[73]: m_eye = np.eye(4)
      print(m_eye)

      # Ранг такой матрицы равен количеству ее столбцов (или строк),
      # в нашем случае ранг будет равен четырем, для его вычисления
      # на Python воспользуемся функцией matrix_rank():

      rank = np.linalg.matrix_rank(m_eye)
      print(rank)
      print("-"*20)

      # Если мы приравняем элемент в нижнем правом углу к нулю,
      # то ранг станет равен трем:

      m_eye[3][3] = 0
      print(m_eye)

      rank = np.linalg.matrix_rank(m_eye)
      print(rank)

[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
4
-----
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 0.]]
3
```

Рисунок 10. Пример 26

Далее были выполнены задания лабораторной работы и индивидуальное задание:

Задание 1. Условие: Создайте массив NumPy размером 3×3, содержащий числа от 1 до 9. Умножьте все элементы массива на 2, а затем замените все элементы больше 10 на 0. Выведите итоговый массив.

```
[4]: import numpy as np

# Создаем массив 3x3 с числами от 1 до 9
array_3x3 = np.arange(1, 10).reshape(3, 3)
print("Исходный массив:")
print(array_3x3)

# Умножаем все элементы на 2
array_multiplied = array_3x3 * 2
print("\nМассив после умножения на 2:")
print(array_multiplied)

# Заменяем элементы больше 10 на 0
array_multiplied[array_multiplied > 10] = 0
print("\nИтоговый массив (элементы > 10 заменены на 0):")
print(array_multiplied)
```

Рисунок 11. Задание 1 – код

Исходный массив:

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
```

Массив после умножения на 2:

```
[[ 2  4  6]
 [ 8 10 12]
 [14 16 18]]
```

Итоговый массив (элементы > 10 заменены на 0):

```
[[ 2  4  6]
 [ 8 10  0]
 [ 0  0  0]]
```

Рисунок 12. Задание 1 – результат

Задание 2. Условие: Создайте массив NumPy из 20 случайных целых чисел от 1 до 100. Найдите и выведите все элементы, которые делятся на 5 без остатка. Затем замените их на -1 и выведите обновленный массив.

```
# Создаем массив из 20 случайных целых чисел от 1 до 100
random_array = np.random.randint(1, 101, size=20)
print("Исходный массив:")
print(random_array)

# Находим элементы, делящиеся на 5 без остатка
mask_div_by_5 = random_array % 5 == 0
divisible_by_5 = random_array[mask_div_by_5]
print("\nЭлементы, делящиеся на 5 без остатка:")
print(divisible_by_5)

# Заменяем их на -1
random_array[mask_div_by_5] = -1
print("\nОбновленный массив (делящиеся на 5 заменены на -1):")
print(random_array)
```

Исходный массив:
[63 32 12 100 49 91 12 88 59 21 31 37 32 6 59 22 42 52
 44 47]

Элементы, делящиеся на 5 без остатка:
[100]

Обновленный массив (делящиеся на 5 заменены на -1):
[63 32 12 -1 49 91 12 88 59 21 31 37 32 6 59 22 42 52 44 47]

Рисунок 13. Задание 2

Задание 3.

Условие: Создайте два массива NumPy размером 1×5, заполненные случайными числами от 0 до 50. Объедините эти массивы в один двумерный массив (по строкам). Разделите полученный массив на два массива, каждый из которых содержит 5 элементов. Выведите все промежуточные и итоговые результаты.

```
[6]: # Создаем два массива 1x5 со случайными числами от 0 до 50
array1 = np.random.randint(0, 51, size=(1, 5))
array2 = np.random.randint(0, 51, size=(1, 5))

print("Первый массив (1x5):")
print(array1)
print("\nВторой массив (1x5):")
print(array2)

# Объединяем массивы по строкам (вертикальное объединение)
merged_array = np.vstack((array1, array2))
print("\nОбъединенный массив (по строкам):")
print(merged_array)
print(f"Форма объединенного массива: {merged_array.shape}")

# Разделяем полученный массив на два массива по 5 элементов каждый
# Вариант 1: разделение по строкам (каждая строка - отдельный массив)
split_arrays = np.vsplit(merged_array, 2)
print("\nРазделение по строкам (каждый массив содержит 5 элементов):")
print("Первый массив:")
```

Рисунок 14. Задание 3 – код

Первый массив (1x5):
[[6 33 50 36 34]]

Второй массив (1x5):
[[29 44 23 42 24]]

Объединенный массив (по строкам):
[[6 33 50 36 34]
[29 44 23 42 24]]

Форма объединенного массива: (2, 5)

Разделение по строкам (каждый массив содержит 5 элементов):
Первый массив:
[[6 33 50 36 34]]

Второй массив:
[[29 44 23 42 24]]

Альтернативный способ разделения:
Первый массив (первая строка):
[[6 33 50 36 34]]

Второй массив (вторая строка):
[[29 44 23 42 24]]

Рисунок 15. Задание 3 – результат

Задание 4. Условие: Создайте массив из 50 чисел, равномерно распределенных от -10 до 10. Вычислите сумму всех элементов, сумму положительных элементов и сумму отрицательных элементов. Выведите результаты.

```
# Создаем массив из 50 чисел, равномерно распределенных от -10 до 10
linear_array = np.linspace(-10, 10, 50)
print("Массив из 50 чисел, равномерно распределенных от -10 до 10:")
print(linear_array)
print(f"\nКоличество элементов: {len(linear_array)}")
print(f"Первый элемент: {linear_array[0]:.2f}")
print(f"Последний элемент: {linear_array[-1]:.2f}")

# Вычисляем сумму всех элементов
total_sum = np.sum(linear_array)
print(f"\nСумма всех элементов: {total_sum:.4f}")

# Вычисляем сумму положительных элементов
positive_mask = linear_array > 0
positive_sum = np.sum(linear_array[positive_mask])
print(f"Сумма положительных элементов: {positive_sum:.4f}")

# Вычисляем сумму отрицательных элементов
negative_mask = linear_array < 0
negative_sum = np.sum(linear_array[negative_mask])
print(f"Сумма отрицательных элементов: {negative_sum:.4f}")
```

Рисунок 16. Задание 4 – код

Массив из 50 чисел, равномерно распределенных от -10 до 10:

-10.	-9.59183673	-9.18367347	-8.7755102	-8.36734694
-7.95918367	-7.55102041	-7.14285714	-6.73469388	-6.32653061
-5.91836735	-5.51020408	-5.10204082	-4.69387755	-4.28571429
-3.87755102	-3.46938776	-3.06122449	-2.65306122	-2.24489796
-1.83673469	-1.42857143	-1.02040816	-0.6122449	-0.20408163
0.20408163	0.6122449	1.02040816	1.42857143	1.83673469
2.24489796	2.65306122	3.06122449	3.46938776	3.87755102
4.28571429	4.69387755	5.10204082	5.51020408	5.91836735
6.32653061	6.73469388	7.14285714	7.55102041	7.95918367
8.36734694	8.7755102	9.18367347	9.59183673	10.

Количество элементов: 50

Первый элемент: -10.00

Последний элемент: 10.00

Сумма всех элементов: 0.0000

Сумма положительных элементов: 127.5510

Сумма отрицательных элементов: -127.5510

Рисунок 17. Задание 4 – результат

Задание 5. Условие: Создайте единичную матрицу размером 4×4. Создайте диагональную матрицу размером 4×4 с диагональными элементами [5, 10, 15, 20] (не использовать циклы). Найдите сумму всех элементов каждой из этих матриц и сравните результаты.

```
# Создаем единичную матрицу 4x4
identity_matrix = np.eye(4)
print("Единичная матрица 4x4:")
print(identity_matrix)

# Создаем диагональную матрицу 4x4 с диагональными элементами [5, 10, 15, 20]
diagonal_elements = np.array([5, 10, 15, 20])
diagonal_matrix = np.diag(diagonal_elements)
print("\nДиагональная матрица 4x4:")
print(diagonal_matrix)

# Находим сумму всех элементов каждой матрицы
sum_identity = np.sum(identity_matrix)
sum_diagonal = np.sum(diagonal_matrix)

print(f"\nСумма всех элементов единичной матрицы: {sum_identity}")
print(f"Сумма всех элементов диагональной матрицы: {sum_diagonal}")

# Сравниваем результаты
print(f"\nСравнение результатов:")
print(f"Сумма диагональной матрицы больше суммы единичной на {sum_diagonal - sum_identity}")
print(f"Сумма диагональной матрицы в {sum_diagonal / sum_identity:.2f} раз больше суммы единичной")
```

Рисунок 18. Задание 5 – код

Единичная матрица 4x4:

```
[[1. 0. 0. 0.]  
 [0. 1. 0. 0.]  
 [0. 0. 1. 0.]  
 [0. 0. 0. 1.]]
```

Диагональная матрица 4x4:

```
[[ 5  0  0  0]  
 [ 0 10  0  0]  
 [ 0  0 15  0]  
 [ 0  0  0 20]]
```

Сумма всех элементов единичной матрицы: 4.0

Сумма всех элементов диагональной матрицы: 50

Сравнение результатов:

Сумма диагональной матрицы больше суммы единичной на 46.0

Сумма диагональной матрицы в 12.50 раз больше суммы единичной

Рисунок 19. Задание 5 – результат

Задание 6. Условие: Создайте две квадратные матрицы NumPy размером 3×3, заполненные случайными целыми числами от 1 до 20. Вычислите и выведите их сумму, разность и поэлементное произведение.

```
# Создаем две матрицы 3x3 со случайными целыми числами от 1 до 20  
matrix_a = np.random.randint(1, 21, size=(3, 3))  
matrix_b = np.random.randint(1, 21, size=(3, 3))
```

```
print("Первая матрица A (3x3):")  
print(matrix_a)  
print("\nВторая матрица B (3x3):")  
print(matrix_b)
```

```
# Вычисляем сумму матриц  
sum_matrix = matrix_a + matrix_b  
print("\n1. Сумма матриц (A + B):")  
print(sum_matrix)
```

```
# Вычисляем разность матриц  
diff_matrix = matrix_a - matrix_b  
print("\n2. Разность матриц (A - B):")  
print(diff_matrix)
```

```
# Вычисляем поэлементное произведение  
elementwise_product = matrix_a * matrix_b  
print("\n3. Поэлементное произведение (A * B):")  
print(elementwise_product)
```

Рисунок 20. Задание 6 – код

```

Первая матрица A (3x3):
[[ 6 11  8]
 [ 3  6  6]
 [ 2 15  7]]

Вторая матрица B (3x3):
[[ 4 20  2]
 [ 3  2 17]
 [10  8 17]]

1. Сумма матриц (A + B):
[[10 31 10]
 [ 6  8 23]
 [12 23 24]]

2. Разность матриц (A - B):
[[ 2 -9  6]
 [ 0  4 -11]
 [-8  7 -10]]

3. Поэлементное произведение (A * B):
[[ 24 220 16]
 [ 9 12 102]
 [ 20 120 119]]

```

Рисунок 21. Задание 6 – результат

Задание 7. Условие: Создайте две матрицы NumPy: первую размером 2×3, заполненную случайными числами от 1 до 10, и вторую размером 3×2, заполненную случайными числами от 1 до 10. Выполните матричное умножение (@ или np.dot) и выведите результат.

```

# Создаем матрицу A размером 2x3 со случайными числами от 1 до 10
matrix_a = np.random.randint(1, 11, size=(2, 3))
print("Матрица A (2x3):")
print(matrix_a)
print(f"Размерность A: {matrix_a.shape}")

# Создаем матрицу B размером 3x2 со случайными числами от 1 до 10
matrix_b = np.random.randint(1, 11, size=(3, 2))
print("\nМатрица B (3x2):")
print(matrix_b)
print(f"Размерность B: {matrix_b.shape}")

# Выполняем матричное умножение (способ 1 - с помощью @)
result_at = matrix_a @ matrix_b
print("\nРезультат матричного умножения (A @ B):")
print(result_at)
print(f"Размерность результата: {result_at.shape}")

# Выполняем матричное умножение (способ 2 - с помощью np.dot)
result_dot = np.dot(matrix_a, matrix_b)
print("\nРезультат матричного умножения (np.dot(A, B)):")
print(result_dot)
print(f"Размерность результата: {result_dot.shape}")

```

Рисунок 22. Задание 7 – код

```
Матрица A (2x3):  
[[1 6 4]  
 [8 3 3]]  
Размерность A: (2, 3)  
  
Матрица B (3x2):  
[[ 7 10]  
 [ 4  7]  
 [ 2  7]]  
Размерность B: (3, 2)  
  
Результат матричного умножения (A @ B):  
[[ 39  80]  
 [ 74 122]]  
Размерность результата: (2, 2)  
  
Результат матричного умножения (np.dot(A, B)):  
[[ 39  80]  
 [ 74 122]]  
Размерность результата: (2, 2)  
  
Результаты совпадают: True
```

Рисунок 23. Задание 7 – результат

Задание 8. Условие: Создайте случайную квадратную матрицу 3×3. Найдите и выведите определитель этой матрицы и обратную матрицу (если существует, иначе выведите сообщение, что матрица вырождена).

```
# Создаем случайную квадратную матрицу 3x3  
matrix = np.random.randint(1, 10, size=(3, 3))  
print("Случайная квадратная матрица 3x3:")  
print(matrix)  
print(f"Размерность матрицы: {matrix.shape}")  
  
# Вычисляем определитель матрицы  
determinant = np.linalg.det(matrix)  
print(f"\nОпределитель матрицы: {determinant:.4f}")  
  
# Проверяем, является ли матрица вырожденной (определитель ≈ 0)  
if np.isclose(determinant, 0):  
    print("\nМатрица вырождена (определитель ≈ 0).")  
    print("Обратная матрица не существует.")  
else:  
    print("\nМатрица невырождена. Обратная матрица существует.")  
  
# Вычисляем обратную матрицу  
inverse_matrix = np.linalg.inv(matrix)  
print("\nОбратная матрица:")  
print(inverse_matrix)
```

Рисунок 24. Задание 8 – код

```

Случайная квадратная матрица 3x3:
[[3 7 3]
 [6 9 2]
 [2 1 4]]
Размерность матрицы: (3, 3)

Определитель матрицы: -74.0000

Матрица невырождена. Обратная матрица существует.

Обратная матрица:
[[-0.45945946  0.33783784  0.17567568]
 [ 0.27027027 -0.08108108 -0.16216216]
 [ 0.16216216 -0.14864865  0.2027027 ]]

Проверка: A * A^(-1) = I
[[ 1.00000000e+00  1.11022302e-16  0.00000000e+00]
 [-2.22044605e-16  1.00000000e+00  1.11022302e-16]
 [ 0.00000000e+00  1.11022302e-16  1.00000000e+00]]

Должна получиться единичная матрица:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

Результат проверки: True

```

Рисунок 25. Задание 8 – результат

Задание 9. Условие: Создайте матрицу NumPy размером 4x4, содержащую случайные целые числа от 1 до 50. Выведите исходную матрицу, транспонированную матрицу и след матрицы (сумму элементов на главной диагонали). Используйте `np.trace` для нахождения следа.

```

# Создаем матрицу 4x4 со случайными целыми числами от 1 до 50
matrix = np.random.randint(1, 51, size=(4, 4))
print("Исходная матрица (4x4):")
print(matrix)
print(f"Размерность матрицы: {matrix.shape}")

# Транспонируем матрицу
transposed_matrix = matrix.T
print("\nТранспонированная матрица:")
print(transposed_matrix)
print(f"Размерность транспонированной матрицы: {transposed_matrix.shape}")

# Находим след матрицы (сумма элементов на главной диагонали)
trace_value = np.trace(matrix)
print(f"\nСлед матрицы (сумма элементов на главной диагонали): {trace_value}")

```

Рисунок 26. Задание 9 – код

Исходная матрица (4x4):

```
[ [ 2 22 12 14]
  [ 6 40 37 23]
  [28 11 38  5]
  [34 18 40 21]]
```

Размерность матрицы: (4, 4)

Транспонированная матрица:

```
[ [ 2  6 28 34]
  [22 40 11 18]
  [12 37 38 40]
  [14 23  5 21]]
```

Размерность транспонированной матрицы: (4, 4)

След матрицы (сумма элементов на главной диагонали): 101

Рисунок 27. Задание 9 – результат

Решите систему линейных уравнений вида:

$$\begin{cases} 2x + 3y - z = 5 \\ 4x - y + 2z = 6 \\ -3x + 5y + 4z = -2 \end{cases}$$

Задание 10.

```
# Матрица коэффициентов
matrix_a = np.array([
    [2, 3, -1],
    [4, -1, 2],
    [-3, 5, 4]
])

# Вектор правой части
vector_b = np.array([5, 6, -2])

# Решение системы
solution = np.linalg.solve(matrix_a, vector_b)

print("Решение системы:")
print(f"x = {solution[0]:.2f}")
print(f"y = {solution[1]:.2f}")
print(f"z = {solution[2]:.2f}")
```

Рисунок 28. Задание 10 – код

Решение системы:

x = 1.64

y = 0.58

z = 0.01

Рисунок 29. Задание 10 – результат

ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ. Для создания нового металлического сплава используются три вида металлов. В первом металле содержится 60% железа, во втором — 70%, в третьем — 80%. Нужно получить 100 кг сплава с 72% железа. При этом второго металла должно быть в два раза больше, чем первого. Сколько килограммов каждого металла нужно взять?

Матричный метод

```
# Матрица коэффициентов
matrix_a = np.array([
    [1, 1, 1],
    [0.6, 0.7, 0.8],
    [-2, 1, 0]
])

# Вектор правой части
vector_b = np.array([100, 72, 0])

# Решение системы
solution_matrix = np.linalg.solve(matrix_a, vector_b)

print("Матричный метод:")
print(f"x = {solution_matrix[0]:.2f} кг")
print(f"y = {solution_matrix[1]:.2f} кг")
print(f"z = {solution_matrix[2]:.2f} кг")
```

Метод Крамера

```
# Определитель основной матрицы
det_a = np.linalg.det(matrix_a)

# Матрицы для метода Крамера
matrix_x = matrix_a.copy()
matrix_x[:, 0] = vector_b

matrix_y = matrix_a.copy()
matrix_y[:, 1] = vector_b

matrix_z = matrix_a.copy()
matrix_z[:, 2] = vector_b

# Вычисление решений
x = np.linalg.det(matrix_x) / det_a
y = np.linalg.det(matrix_y) / det_a
z = np.linalg.det(matrix_z) / det_a

print("\nМетод Крамера:")
print(f"x = {x:.2f} кг")
print(f"y = {y:.2f} кг")
print(f"z = {z:.2f} кг")
```

Рисунок 30-31. Код для индивидуального задания. Решение матричным методом и методом Крамера


```
Метод Крамера:  
x = 20.00 кг  
y = 40.00 кг  
z = 40.00 кг  
  
Матричный метод:  
x = 20.00 кг  
y = 40.00 кг  
z = 40.00 кг
```

Рисунок 32. Результат выполнения кода для индивидуального задания, решения методом Крамера и матричным методом

Ответы на контрольные вопросы:

1. Каково назначение библиотеки NumPy?

Библиотека NumPy предназначена для эффективной работы с числовыми данными в языке Python. Она предоставляет быстрые многомерные массивы, операции линейной алгебры, статистические функции, средства генерации случайных чисел, векторизованные вычисления и инструменты для обработки больших массивов данных. NumPy является базовой библиотекой для многих средств анализа данных и научных вычислений, например SciPy, pandas и scikit-learn.

2. Что такое массивы ndarray?

ndarray - это основной тип данных библиотеки NumPy, представляющий собой N-мерный однородный массив. Все элементы такого массива обычно имеют один тип данных, например int64 или float64, что позволяет хранить и обрабатывать их быстрее, чем стандартные списки Python. Основные характеристики ndarray: shape - форма массива, ndim - число измерений, size - количество элементов, dtype - тип элементов.

3. Как осуществляется доступ к частям многомерного массива?

Доступ к частям многомерного массива осуществляется с помощью индексации и срезов по каждой оси. Например, для двумерного массива A запись A[i, j] обращается к элементу i-й строки и j-го столбца, A[i, :] - ко всей

i-й строке, $A[:, j]$ - ко всему j-му столбцу, $A[1:3, 0:2]$ - к подматрице. Также применяются отрицательные индексы, шаги среза, многоточие $A[..., 0]$ и добавление новой оси через `np.newaxis`.

4. Как осуществляется расчет статистик по данным?

Расчет статистик выполняется с помощью функций NumPy или методов объекта `ndarray`. Основные функции: `np.mean()` - среднее значение, `np.median()` - медиана, `np.std()` - стандартное отклонение, `np.var()` - дисперсия, `np.min()` и `np.max()` - минимум и максимум, `np.sum()` — сумма, `np.percentile()` и `np.quantile()` - процентиля и квантили. Параметр `axis` позволяет рассчитывать статистики по строкам, столбцам или другим измерениям массива.

5. Как выполняется выборка данных из массивов ndarray?

Выборка данных из `ndarray` выполняется с помощью срезов, целочисленной индексации и логических масок. Например, $A[0:3]$ выбирает первые три элемента или строки, $A[[0, 2, 4]]$ выбирает элементы с указанными индексами, $A[A > 0]$ возвращает только положительные элементы. Для более сложной выборки используются `np.where()`, `np.take()`, `np.extract()`, а также комбинирование условий с операторами `&`, `|` и `~`.

6. Приведите основные виды матриц и векторов. Опишите способы их создания в языке Python.

К основным видам векторов относятся строка-вектор, столбец-вектор, нулевой вектор и единичный вектор. К основным видам матриц относятся квадратные, прямоугольные, нулевые, единичные, диагональные, скалярные, верхнетреугольные, нижнетреугольные, симметрические и кососимметрические матрицы.

В Python с использованием NumPy они создаются функциями: `np.array()` - из списков, `np.zeros()` - нулевая матрица, `np.ones()` - матрица из единиц, `np.eye()` или `np.identity()` - единичная матрица, `np.diag()` - диагональная матрица, `np.arange()` и `np.linspace()` - массивы с последовательностями чисел, `np.reshape()` - изменение формы массива,

`np.triu()` и `np.tril()` - верхняя и нижняя треугольные части матрицы, `np.random` - случайные матрицы. Например, строка-вектор создается как `np.array([1, 2, 3])`, а столбец-вектор - как `np.array([1, 2, 3]).reshape(-1, 1)`.

7. Как выполняется транспонирование матриц?

Транспонирование матрицы - это операция замены строк матрицы ее столбцами. Если дана матрица A , то транспонированная матрица обозначается A^T , а ее элементы определяются правилом: $(A^T)_{ij} = A_{ji}$. В NumPy транспонирование двумерной матрицы выполняется с помощью `A.T`, `A.transpose()` или `np.transpose(A)`.

8. Приведите свойства операции транспонирования матриц.

Основные свойства транспонирования: $(A^T)^T = A$; $(A + B)^T = A^T + B^T$; $(kA)^T = kA^T$; $(AB)^T = B^T A^T$; $\det(A^T) = \det(A)$; $\text{rank}(A^T) = \text{rank}(A)$. Если матрица симметрическая, то $A^T = A$, а если кососимметрическая, то $A^T = -A$. Для обратной матрицы также выполняется $(A^T)^{-1} = (A^{-1})^T$.

9. Какие имеются средства в библиотеке NumPy для выполнения транспонирования матриц?

В NumPy для транспонирования используются атрибут `A.T`, метод `A.transpose()`, функция `np.transpose(A)`, а также функция `np.swapaxes(A, axis1, axis2)`, которая меняет местами две оси многомерного массива. Для массивов с числом измерений больше двух можно задавать порядок осей через `np.transpose(A, axes=...)`. Для одномерного массива `A.T` форму не меняет, поэтому вектор-столбец необходимо задавать как двумерный массив, например через `reshape(-1, 1)`.

10. Какие существуют основные действия над матрицами?

К основным действиям над матрицами относятся умножение матрицы на число, сложение и вычитание матриц, умножение матриц, транспонирование, нахождение определителя, вычисление обратной матрицы, нахождение ранга и следа матрицы, возведение матрицы в степень, решение систем линейных уравнений и применение элементарных преобразований строк или столбцов.

11. Как осуществляется умножение матрицы на число?

Умножение матрицы на число выполняется поэлементно: каждый элемент матрицы умножается на заданный скаляр. Если $B = kA$, то $b_{ij} = k \cdot a_{ij}$. В NumPy такая операция выполняется выражением $k * A$ или $A * k$, также можно использовать функцию `np.multiply(A, k)`.

12. Какие свойства операции умножения матрицы на число?

Операция умножения матрицы на число обладает следующими свойствами: $k(A + B) = kA + kB$; $(k + m)A = kA + mA$; $k(mA) = (km)A$; $1 \cdot A = A$; $0 \cdot A = O$, где O — нулевая матрица; $(-1)A = -A$. Также выполняется свойство совместимости с транспонированием: $(kA)^T = kA^T$.

13. Как осуществляется операция сложения и вычитания матриц?

Сложение и вычитание матриц выполняется поэлементно и возможно для матриц одинакового размера. Если $C = A + B$, то $c_{ij} = a_{ij} + b_{ij}$; если $C = A - B$, то $c_{ij} = a_{ij} - b_{ij}$. В NumPy эти операции выполняются с помощью операторов $A + B$ и $A - B$. При работе с массивами NumPy также возможно broadcasting-расширение размеров, но в классической матричной алгебре размеры матриц должны совпадать.

14. Каковы свойства операций сложения и вычитания матриц?

Сложение матриц обладает коммутативностью $A + B = B + A$, ассоциативностью $(A + B) + C = A + (B + C)$, имеет нейтральный элемент $A + O = A$ и противоположный элемент $A + (-A) = O$. Вычитание можно рассматривать как сложение с противоположной матрицей: $A - B = A + (-B)$. В общем случае вычитание не является коммутативным и ассоциативным. Также выполняется распределительное свойство относительно умножения на число: $k(A \pm B) = kA \pm kB$.

15. Какие имеются средства в библиотеке NumPy для выполнения операций сложения и вычитания матриц?

В NumPy сложение и вычитание матриц выполняются операторами $+$ и $-$, а также функциями `np.add(A, B)` и `np.subtract(A, B)`. Для изменения массива на месте могут использоваться операции $A += B$ и $A -= B$. Если формы

массивов совместимы, NumPy применяет механизм broadcasting, но для обычного сложения и вычитания матриц их размеры должны быть одинаковыми.

16. Как осуществляется операция умножения матриц?

Умножение матриц возможно, если число столбцов первой матрицы равно числу строк второй. Если A имеет размер $m \times n$, а B имеет размер $n \times r$, то результат $C = AB$ имеет размер $m \times r$, а элементы вычисляются по формуле $c_{ij} = \sum a_{ik}b_{kj}$, где суммирование выполняется по k от 1 до n . В NumPy матричное умножение выполняется оператором `@`, например $C = A @ B$. Оператор `*` в NumPy означает поэлементное умножение, а не матричное.

17. Каковы свойства операции умножения матриц?

Умножение матриц обладает ассоциативностью: $(AB)C = A(BC)$, и дистрибутивностью: $A(B + C) = AB + AC$, $(A + B)C = AC + BC$. Оно совместимо с умножением на число: $k(AB) = (kA)B = A(kB)$. В общем случае умножение матриц некоммутативно, то есть $AB \neq BA$. Также выполняются свойства $AI = IA = A$, $AO = O$, $(AB)^T = B^T A^T$, а для квадратных матриц $\det(AB) = \det(A)\det(B)$.

18. Какие имеются средства в библиотеке NumPy для выполнения операции умножения матриц?

Для матричного умножения в NumPy используются оператор `@`, функция `np.matmul(A, B)`, функция `np.dot(A, B)` для двумерных массивов, а также метод `A.dot(B)`. Для последовательного перемножения нескольких матриц удобно применять `np.linalg.multi_dot()`. Для более сложных тензорных вычислений могут использоваться `np.tensordot()` и `np.einsum()`.

19. Что такое определитель матрицы? Каковы свойства определителя матрицы?

Определитель - это числовая характеристика квадратной матрицы, обозначаемая $\det(A)$ или $|A|$. Он показывает, во сколько раз линейное преобразование, заданное матрицей, изменяет ориентированный объем, а также позволяет определить, является ли матрица вырожденной. Если $\det(A)$

$= 0$, то матрица вырождена и обратной матрицы не имеет; если $\det(A) \neq 0$, то матрица невырождена.

Основные свойства определителя: $\det(I) = 1$; $\det(A^T) = \det(A)$; $\det(AB) = \det(A)\det(B)$; при перестановке двух строк или столбцов знак определителя меняется; если две строки или два столбца равны или пропорциональны, то $\det(A) = 0$; умножение строки на число k умножает определитель на k ; прибавление к строке другой строки, умноженной на число, определитель не изменяет; для матрицы порядка n выполняется $\det(kA) = k^n \det(A)$; $\det(A^{-1}) = 1 / \det(A)$, если обратная матрица существует.

20. Какие имеются средства в библиотеке NumPy для нахождения значения определителя матрицы?

В NumPy определитель квадратной матрицы вычисляется функцией `np.linalg.det(A)`. Для численно устойчивого получения знака и логарифма модуля определителя используется `np.linalg.slogdet(A)`. Так как вычисления выполняются в числах с плавающей точкой, результат для целочисленных матриц может иметь небольшую погрешность, поэтому при необходимости его округляют, например `np.round(np.linalg.det(A))`.

21. Что такое обратная матрица? Какой алгоритм нахождения обратной матрицы?

Обратная матрица A^{-1} — это такая матрица, для которой выполняется $AA^{-1} = A^{-1}A = I$, где I - единичная матрица. Обратная матрица существует только для квадратной невырожденной матрицы, то есть при $\det(A) \neq 0$.

Один из алгоритмов нахождения обратной матрицы основан на методе Гаусса - Жордана: сначала проверяют, что матрица квадратная и $\det(A) \neq 0$; затем составляют расширенную матрицу $[A \mid I]$; с помощью элементарных преобразований строк приводят левую часть к единичной матрице; после этого правая часть становится обратной матрицей A^{-1} . Также обратную матрицу можно находить по формуле $A^{-1} = \text{adj}(A) / \det(A)$, где $\text{adj}(A)$ - присоединенная матрица.

22. Каковы свойства обратной матрицы?

Основные свойства обратной матрицы: обратная матрица единственна; $(A^{-1})^{-1} = A$; $(AB)^{-1} = B^{-1}A^{-1}$; $(A^T)^{-1} = (A^{-1})^T$; $(kA)^{-1} = (1/k)A^{-1}$ при $k \neq 0$; $\det(A^{-1}) = 1 / \det(A)$; если A обратима, то $\text{rank}(A)$ равен ее порядку. Матрица имеет обратную тогда и только тогда, когда она квадратная и ее определитель не равен нулю.

23. Какие имеются средства в библиотеке NumPy для нахождения обратной матрицы?

В NumPy обратная матрица находится функцией `np.linalg.inv(A)`. Также ее можно получить решением матричного уравнения $AX = I$ с помощью `np.linalg.solve(A, I)`, где $I = \text{np.eye}(n)$. Для плохо обусловленных или вырожденных матриц применяют псевдообратную матрицу `np.linalg.pinv(A)`. Перед вычислением обратной матрицы полезно проверять определитель, ранг или число обусловленности `np.linalg.cond(A)`.

24. Самостоятельно изучите метод Крамера для решения систем линейных уравнений. Приведите алгоритм решения системы линейных уравнений методом Крамера средствами библиотеки NumPy.

Метод Крамера применяется для решения системы линейных уравнений $AX = b$, где A - квадратная матрица коэффициентов, X — вектор неизвестных, b - вектор свободных членов. Метод применим, если главный определитель $D = \det(A)$ не равен нулю. Для каждой неизвестной x_i формируется матрица A_i , в которой i -й столбец матрицы A заменяется вектором b . Затем неизвестная находится по формуле $x_i = \det(A_i) / \det(A)$.

Алгоритм средствами NumPy:

```
import numpy as np

A = np.array([[2, 1, -1],
              [-3, -1, 2],
              [-2, 1, 2]], dtype=float)
b = np.array([8, -11, -3], dtype=float)

D = np.linalg.det(A)
if np.isclose(D, 0):
    raise ValueError('Метод Крамера неприменим: det(A) = 0')

x = np.empty(len(b))
```

```

for i in range(len(b)):
    Ai = A.copy()
    Ai[:, i] = b
    x[i] = np.linalg.det(Ai) / D

print(x)

```

Для больших систем метод Крамера используется редко, так как он требует вычисления нескольких определителей и менее эффективен, чем методы на основе разложения матриц или функция `np.linalg.solve()`.

25. Самостоятельно изучите матричный метод для решения систем линейных уравнений. Приведите алгоритм решения системы линейных уравнений матричным методом средствами библиотеки NumPy.

Матричный метод решения системы линейных уравнений основан на записи системы в виде $AX = b$. Если матрица A квадратная и невырожденная, то существует обратная матрица A^{-1} , и решение находится по формуле $X = A^{-1}b$.

Алгоритм средствами NumPy:

```

import numpy as np

A = np.array([[2, 1, -1],
              [-3, -1, 2],
              [-2, 1, 2]], dtype=float)
b = np.array([8, -11, -3], dtype=float)

if np.isclose(np.linalg.det(A), 0):
    raise ValueError('Матрица A вырождена, обратной матрицы нет')

A_inv = np.linalg.inv(A)
x = A_inv @ b

print(x)

```

На практике вместо явного вычисления обратной матрицы чаще используют функцию `np.linalg.solve(A, b)`, так как она быстрее и численно устойчивее: `x = np.linalg.solve(A, b)`.